

SDN-Lab2

Haochen You

April 2026

1 Statement of Work

Warm-up:

Try the two simple networks with FRR switches in the lab's repository.

Edit the configuration of the switches to enable Equal Cost Multi-Path (ECMP).

Use route policies to reroute traffic in the toy networks.

Task:

Construct a fattree (k=4) topology using Mininet (frrnet) Python API

Make sure all hosts are reachable, and ECMP is correctly established.

If there are issues, try to solve it.

(Optional) Try to reuse the ASNs across the switches in the same layer and the same pod.

2 Preparation

clone 仓库, 运行 example/main.py, 检查环境是否正常:

```
[sdn@archlinux ~]$ cd sdn-lab2
[sdn@archlinux sdn-lab2]$ sudo uv run example/main.py
Using CPython 3.14.3 interpreter at: /usr/bin/python3.14
Creating virtual environment at: .venv
Installed 3 packages in 98ms
*** Creating network
*** Adding hosts:
h1 h2
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s2) (10.00Mbit 10ms delay) (10.00Mbit 10ms delay) (s1, s2)
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 2 switches
s1 s2
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

图 1: Preparation

3 Warm-up

3.1 对于 BGP 路由交换的基本认知

我们启动 example/main.py 网络拓扑之后，在 s1 的 Ethernet1-1 接口上启动 tcpdump 后台抓包（因为在 mininet 的虚拟网络命名空间里，需要自行安装），过滤 BGP 协议。之后，我们重置 s1 的 BGP 会话，使其重新与 s2 建立连接并交换路由，等待约 10s 之后，使用 pingall 验证路由恢复正常，最后停止抓包，用 wireshark 查看抓包情况如下（筛选 BGP 之后的结果）。

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.1	10.0.0.0	BGP	85	KEEPALIVE Message
3	0.345735	10.0.0.0	10.0.0.1	BGP	85	KEEPALIVE Message
5	3.001211	10.0.0.1	10.0.0.0	BGP	85	KEEPALIVE Message
7	3.236153	10.0.0.0	10.0.0.1	BGP	85	KEEPALIVE Message
9	5.565043	10.0.0.0	10.0.0.1	BGP	89	NOTIFICATION Message
19	7.598966	10.0.0.0	10.0.0.1	BGP	197	OPEN Message
22	7.609911	10.0.0.1	10.0.0.0	BGP	197	OPEN Message
23	7.609918	10.0.0.1	10.0.0.0	BGP	197	OPEN Message
24	7.609922	10.0.0.1	10.0.0.0	BGP	87	NOTIFICATION Message
29	7.620803	10.0.0.0	10.0.0.1	BGP	197	OPEN Message
30	7.620803	10.0.0.0	10.0.0.1	BGP	87	NOTIFICATION Message
35	7.631064	10.0.0.1	10.0.0.0	BGP	85	KEEPALIVE Message
36	7.641831	10.0.0.0	10.0.0.1	BGP	85	KEEPALIVE Message
39	8.847556	10.0.0.0	10.0.0.1	BGP	144	UPDATE Message, UPDATE Message
41	8.864368	10.0.0.1	10.0.0.0	BGP	144	UPDATE Message, UPDATE Message
43	8.948394	10.0.0.1	10.0.0.0	BGP	118	UPDATE Message
44	8.948408	10.0.0.0	10.0.0.1	BGP	118	UPDATE Message
47	10.632707	10.0.0.1	10.0.0.0	BGP	85	KEEPALIVE Message
49	10.643389	10.0.0.0	10.0.0.1	BGP	85	KEEPALIVE Message

图 2: BGP

我们在图中能够看到三种不同的报文，分别是 OPEN（协商 BGP 连接参数），KeepAlive（确认连接建立并周期性会话），Update（携带路由信息的核心报文）。上图也验证了 BGP 路由交换的完成过程——先通过 OPEN 建立 TCP 连接和 BGP 绘画，再通过 Update 报文宣告各自的网络，最终双方都获得了互相的信息，实现了互通。

3.2 运行 Warmup 网络，观察分析

接下来，我们运行 warmup/main.py，简单运行 pingall、s1 show ip bgp、h1 traceroute h2，观察。

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> ^[Ah2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> s1 show ip bgp
BGP table version is 2, local router ID is 192.168.1.1, vrf id 0
Default local pref 100, local AS 1
Status codes: s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes: i - IGP, e - EGP, ? - Incomplete
RPKI validation codes: V valid, I invalid, N Not found

   Network          Next Hop              Metric LocPrf Weight Path
*> 192.168.1.0/24    0.0.0.0(archlinux)
                                0         32768 i
*> 192.168.2.0/24    10.0.13.1(archlinux)
                                0 3 2 i
*                   10.0.14.1(archlinux)
                                0 4 2 i

Displayed 2 routes and 3 total paths
mininet> h1 traceroute h2
traceroute to 192.168.2.2 (192.168.2.2), 30 hops max, 60 byte packets
 1 192.168.1.1 (192.168.1.1)  0.022 ms  0.002 ms  0.002 ms
 2 10.0.13.1 (10.0.13.1)  20.483 ms  20.474 ms  20.469 ms
 3 * * *
 4 192.168.2.2 (192.168.2.2)  41.501 ms  41.498 ms  41.494 ms
```

图 3: Warmup Net

观察一下 s1 的路由表项，可以发现 s1 有两条到达 192.168.2.0/24 的路径，其中经过 s3 再到 s2 的被

标记为最优路由，另一条则未被选中。我们模拟一下比较的过程：两条路径的 AS-Path 长度都是 2，Local Preference 也相同，所以 BGP 继续往下比较 tie-breaker，最终比到了 Router ID：s3 的 Router ID (10.0.23.1) 比 s4 的 (10.0.24.1) 小，所以 s3 胜出。

接下来再看看 traceroute 的过程，第一跳走到 s1 (h1 的网关)；第二跳走了 s3，即我们上面得到的最优路由；第三跳是 **，这代表 s2 没有回应 ICMP，是正常现象；第四跳走到了 h2。我们发现，即使存在两条等价路径，BGP 默认直选一条最优路由，另一条路径则是闲置的。这代表带宽被严重浪费了，ECMP 则可以帮我们一定程度上解决这个问题。

3.3 启用 ECMP，观察吞吐量的变化

为了证明上面说到的这一点，我们在启用 ECMP 之前，先测试一下 ECMP 开启之前的吞吐量。测量完成之后，我们退出 mininet，编辑配置文件 warmup/config/s1.conf，在 router bgp 1 下面加上 bgp bestpath as-path multipath-relax。s2 同理。之后我们重新启动 warmup 网络，验证 ECMP 生效，验证如下：

```
mininet> s1 show ip bgp
BGP table version is 2, local router ID is 192.168.1.1, vrf id 0
Default local pref 100, local AS 1
Status codes: s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Next hop codes: @NNN next hop's vrf id, < announce-nh-self
Origin codes: i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

   Network        Next Hop        Metric LocPrf Weight Path
*>  192.168.1.0/24  0.0.0.0(archlinux)
                                0         32768 i
*>  192.168.2.0/24  10.0.13.1(archlinux)
                                0         3 2 i
*=  192.168.2.0/24  10.0.14.1(archlinux)
                                0         4 2 i

Displayed 2 routes and 3 total paths
```

图 4: ECMP Work

我们看到，s4 的路由由 * 变为了 *=，这就是 multipath 标记。这代表它被选为了“等价的最优路由”。之后我们再测试吞吐量，下面是开启前后吞吐量的对比：

```
mininet> h1 iperf -s &
mininet> h2 iperf -c h1 -P 4
-----
Client connecting to 192.168.1.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 4] local 192.168.2.2 port 45234 connected with 192.168.1.2 port 5001
[ 3] local 192.168.2.2 port 45250 connected with 192.168.1.2 port 5001
[ 2] local 192.168.2.2 port 45222 connected with 192.168.1.2 port 5001
[ 1] local 192.168.2.2 port 45206 connected with 192.168.1.2 port 5001

[ ID] Interval       Transfer     Bandwidth
[ 1] 0.0000-17.1380 sec  5.10 MBytes  2.50 Mbits/sec
[ 2] 0.0000-17.9373 sec  4.73 MBytes  2.21 Mbits/sec
[ 3] 0.0000-17.9543 sec  3.72 MBytes  1.74 Mbits/sec
[ 4] 0.0000-18.1697 sec  5.22 MBytes  2.41 Mbits/sec
[SUM] 0.0000-18.1698 sec  18.8 MBytes  8.67 Mbits/sec
```

图 5: Basic Instructions

```
mininet> h1 iperf -s &
mininet> h2 iperf -c h1 -P 4
-----
Client connecting to 192.168.1.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 3] local 192.168.2.2 port 37938 connected with 192.168.1.2 port 5001
[ 1] local 192.168.2.2 port 37958 connected with 192.168.1.2 port 5001
[ 2] local 192.168.2.2 port 37926 connected with 192.168.1.2 port 5001
[ 4] local 192.168.2.2 port 37950 connected with 192.168.1.2 port 5001

[ ID] Interval       Transfer     Bandwidth
[ 3] 0.0000-11.5310 sec  11.7 MBytes  8.49 Mbits/sec
[ 4] 0.0000-14.4419 sec  5.96 MBytes  3.46 Mbits/sec
[ 2] 0.0000-14.5579 sec  4.87 MBytes  2.81 Mbits/sec
[ 1] 0.0000-14.9094 sec  4.69 MBytes  2.64 Mbits/sec
[SUM] 0.0000-14.9096 sec  27.2 MBytes  15.3 Mbits/sec
```

图 6: Throughput Comparison

我们可以发现，吞吐量从原来的 8.67Mbps 变为了 15.3Mbps，接近翻倍。这说明 ECMP 生效了。

3.4 配置路由策略，使流量只经过 s4

之后，我们再退出 mininet，warmup/config/s1.conf。修改两处，分别是定义名为 LOWER_LP 的路由策略，把匹配到的路由的 Local Preference 改成 50（默认的是 100）；以及在 router bgp 之下加上 neighbor 10.0.13.1 route-map LOWER_LP in，表示应用上面定义的路由策略，针对从 s3 收到的路由生效。这样从 s3 收到的路由，LP 被改为 50，而从 s4 收到的路由 LP 不变（100），这样 BGP 选路时，LP 高的优先，流量就走 s4 了。s2 的修改同理。

修改完成之后，我们重启 warmup 网络，进入 mininet 之后再执行 s1 show ip bgp 和 h1 traceroute h2 这两条命令，观察结果：

```
mininet> s1 show ip bgp
BGP table version is 2, local router ID is 192.168.1.1, vrf id 0
Default local pref 100, local AS 1
Status codes: s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes: i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

   Network          Next Hop              Metric LocPrf Weight Path
*>  192.168.1.0/24    0.0.0.0(archlinux)
                                     0          32768 i
*>  192.168.2.0/24    10.0.14.1(archlinux)
                                     0          0 4 2 i
*   10.0.13.1(archlinux)
                                     50          0 3 2 i

Displayed 2 routes and 3 total paths
mininet> h1 traceroute h2
traceroute to 192.168.2.2 (192.168.2.2), 30 hops max, 60 byte packets
 1  192.168.1.1 (192.168.1.1)  0.020 ms  0.002 ms  0.001 ms
 2  10.0.14.1 (10.0.14.1)  25.782 ms  25.775 ms  25.770 ms
 3  * * *
 4  192.168.2.2 (192.168.2.2)  46.314 ms  46.311 ms  46.306 ms
```

图 7: After Routing Policy Configuration

我们观察到，s3 路由的 LP 变成了 50，s4 还是默认。所以现在 s4 被标记成了最优 *>，且 ECMP 的 = 标记消失了（因为 LP 不同，不再等价）。所以在 traceroute 之中，h1 到 h2 走的是 s4。

3.5 尝试不同的匹配选项和设置选项

在上面的研究中，我们初步了解了路由策略的执行机理。下面我们来试试其他不同的匹配选项和设置选项。我们把 s1 的 route-map 部分改成 set as-path prepend 3 3 3，这条命令在从 s3 收到的路由的 AS-Path 前面额外插入三个 ASN 3，原来从 s3 来的路由 AS-Path 是 3 2（经过 s3 再到 s2，长度 2），加了 prepend 后变成 3 3 3 3 2（长度 5）。而从 s4 来的路由 AS-Path 还是 4 2（长度 2）。BGP 选路规则第 4 条是 AS-Path 越短越优先，所以最后选择 s4。

我们还是像之前一样，执行命令验证这一点，截图如下：

```

mininet> s1 show ip bgp
BGP table version is 2, local router ID is 192.168.1.1, vrf id 0
Default local pref 100, local AS 1
Status codes: s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes: i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

   Network        Next Hop           Metric LocPrf Weight Path
*> 192.168.1.0/24  0.0.0.0(archlinux)
                                0         32768 i
*> 192.168.2.0/24  10.0.14.1(archlinux)
                                0 4 2 i
*                  10.0.13.1(archlinux)
                                0 3 3 3 2 i

Displayed 2 routes and 3 total paths
mininet> h1 traceroute h2
traceroute to 192.168.2.2 (192.168.2.2), 30 hops max, 60 byte packets
 1 192.168.1.1 (192.168.1.1)  0.037 ms  0.002 ms  0.001 ms
 2 10.0.14.1 (10.0.14.1)  105.533 ms  105.525 ms  105.522 ms
 3 * * *
 4 192.168.2.2 (192.168.2.2)  127.044 ms  127.041 ms  127.038 ms

```

图 8: Prepend ASN

结果和我们预料的相同。s4 路由是最优的，s3 路由未被选中，path 发生了变化，没有出现 ECMP 的 = 标志，traceroute 走的是 s4。

4 Tasks

4.1 定义 k=4 的 fattree 拓扑

我们先给出拓扑图，方便分析：

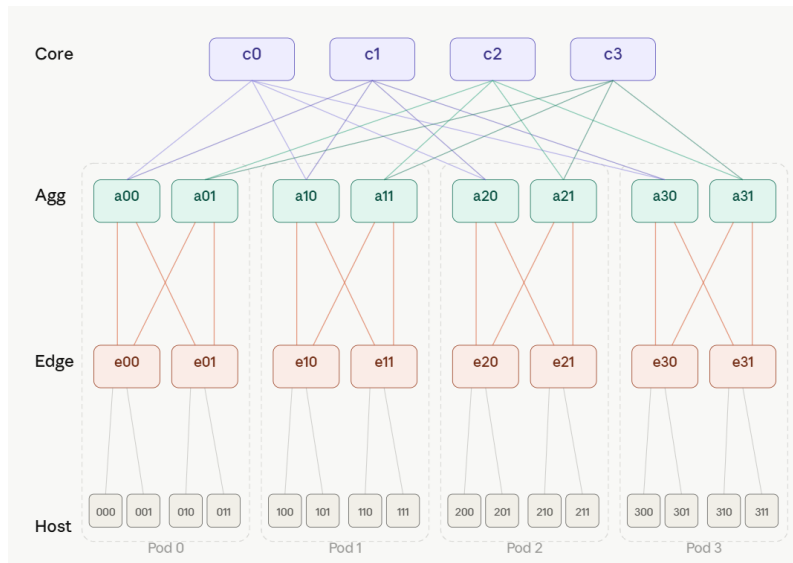


图 9: Topo Graph

下面我们来定义 k=4 的 fattree 拓扑。我们分析一下有什么是需要改的：

首先，基于题目要求，导入的基类改为 `frnet.topo.FrrTopo`，这样 `frnet` 才能自动给交换机加载 FRR 路由套件；其次，给 `addSwitch` 加上 `daemons=["bgpd"]`，没有这个参数，交换机就不会启动 FRR。之后，给 `addLink` 加上接口名、带宽、时延，因为之前二层交换不关心端口名，但现在改为三层路由，配置文件里要用接口名来绑定 IP 地址，所以必须给每个端口起名字。另外，按照题目要求，在交换机间链路设了 `bw=10`, `delay="10ms"`，主机到边缘的链路不设；然后，我们需要 `addHost` 手动指定 IP 和默认路由，在 `Lab1` 里是自动分配的，现在我们手动指定。每台主机独占一个 /24 子网，网关指向它连接的边缘交换机；最后我们修改一下启动方式，现在只需要调用 `frnet_main(FatTree)`，`frnet` 会帮我们处理。

另外，我们设定好接口编号的逻辑。以边缘交换机 `e00` 为例，它有 4 个端口：Ethernet1 连 `a00`（汇聚 0），Ethernet2 连 `a01`（汇聚 1），Ethernet3 连 `h000`（主机 0），Ethernet4 连 `h001`（主机 1）。汇聚和核心交换机也是类似的 4 端口结构。

下面是具体的拓扑代码：

```
1 from frnet import frnet_main
2 from frnet.topo import FrrTopo
3
4
5 class FatTree(FrrTopo):
6     def build(self):
7         k = 4
8         # 创建 4 台核心交换机 (c0, c1, c2, c3)
9         cores = []
10        for i in range(k):
11            cores.append(self.addSwitch(f'c{i}', daemons=["bgpd"]))
12
13        # 遍历 4 个 Pod (p=0,1,2,3)
14        # 每个 Pod 有 2 台汇聚交换机 + 2 台边缘交换机
15        for p in range(k):
16            # 创建汇聚交换机:
17            aggs = []
18            for i in range(k // 2):
19                aggs.append(self.addSwitch(f'a{p}{i}', daemons=["bgpd"]))
20            # 创建边缘交换机:
21            edges = []
22            for i in range(k // 2):
23                edges.append(self.addSwitch(f'e{p}{i}', daemons=["bgpd"]))
24            # 每台边缘交换机连所有汇聚交换机
25            for ei in range(k // 2):
26                for ai in range(k // 2):
27                    self.addLink(
28                        edges[ei], aggs[ai],
29                        intf1=f"Ethernet{ai + 1}",
30                        intf2=f"Ethernet{ei + 1}",
31                        bw=10, delay="10ms"
32                    )
33            # 每台边缘交换机连 2 台主机
34            for ei in range(k // 2):
35                for j in range(k // 2):
36                    subnet_id = ei * 2 + j # 子网编号
37                    host = self.addHost(
38                        f'h{p}{ei}{j}',
```

```

39         ip=f"10.{p}.{subnet_id}.2/24",
40         defaultRoute=f"via 10.{p}.{subnet_id}.1"
41     )
42     self.addLink(
43         edges[ei], host,
44         intf1=f"Ethernet{k // 2 + j + 1}"
45     )
46
47     # 汇聚的 Ethernet3, Ethernet4 连核心
48     # 核心的 Ethernet{p+1} 连该 Pod
49     for ai in range(k // 2):
50         for j in range(k // 2):
51             core_idx = ai * (k // 2) + j
52             self.addLink(
53                 aggs[ai], cores[core_idx],
54                 intf1=f"Ethernet{k // 2 + j + 1}",
55                 intf2=f"Ethernet{p + 1}",
56                 bw=10, delay="10ms"
57             )
58
59 if __name__ == "__main__":
60     frnnet_main(FatTree)

```

给该文件命名为 FattreeTopo.py，这样，我们就按照实验要求建立了拓扑。

4.2 给端口分配 IP，设计 ASN，写 BGP 配置文件

给主机分配 IP 的工作在上一步已完成。每台主机独占一个 /24 子网：10.pod. 子网号.0/24。主机 IP 是.2，网关（即边缘交换机接口）是.1。例如 h000 的 IP 是 10.0.0.2/24，网关 10.0.0.1 配在 e00 的 Ethernet3 上。

关于交换机之间链路 IP 的配置，我使用了 /31 点对点子网，每条链路占两个 IP。具体来说，交换机之间链路 IP 我设置为 172.16.{编号}.0/31。对于编号，具体来说：边缘交换机到汇聚交换机的链路我编号为 0-15；汇聚交换机到核心交换机的链路我编号为 16-31。其中，每条链路两端拥有一个 IP，一段为.0，一端为.1。

关于 ASN 分配，我的方案如下：

核心层：c0=1, c1=2, c2=3, c3=4；

汇聚层：a00=11, a01=12, a10=13, a11=14, a20=15, a21=16, a30=17, a31=18；

边缘层：e00=21, e01=22, e10=23, e11=24, e20=25, e21=26, e30=27, e31=28；

不同层十位数不同，且所有 ASN 都不同，保证全是 eBGP 连接。

最后，关于 BGP 的配置逻辑，每台交换机的配置文件需要下面四部分内容：首先，给每个 Ethernet 端口都要分配 IP 地址；其次，要配置 BGP 的邻居，就是要告诉 BGP 要和谁交换路由（即一条链路上对面端口的 IP 和对面的 ASN）；然后关于 BGP 网络，查询资料发现之后边缘交换机需要宣告主机子网，汇聚交换机和核心交换机是不需要的，因为它们没有直连主机，会自动中继；最后，如有需要，可以选择开启 ECMP（后面的实验步骤会用到）。

接下来，为了实现这些配置，我写了生成 BGP 配置文件的脚本。这个脚本能再 sdn-lab2/task 文件夹下运行，自动在 config 子文件夹生成 20 个交换机的配置文件，脚本如下：

```

1 import os
2 k = 4

```

```

3 config_dir = "task/config"
4 os.makedirs(config_dir, exist_ok=True)
5 # ASN分配
6 def core_asn(ci):          return ci + 1          # 1-4
7 def agg_asn(p, ai):        return 11 + p * 2 + ai  # 11-18
8 def edge_asn(p, ei):       return 21 + p * 2 + ei  # 21-28
9 # 链路IP
10 def edge_agg_link_id(p, ei, ai): return p * 4 + ei * 2 + ai      # 0-15
11 def agg_core_link_id(p, ai, j):  return 16 + p * 4 + ai * 2 + j    # 16-31
12 def link_ip(lid, side):         return f"172.16.{lid}.{side}"
13 # 主机子网
14 def host_network(p, ei, j):
15     sid = ei * 2 + j
16     return f"10.{p}.{sid}.0/24"
17
18 def host_gateway(p, ei, j):
19     sid = ei * 2 + j
20     return f"10.{p}.{sid}.1"
21 # 生成边缘交换机配置
22 def gen_edge(p, ei):
23     lines = ["frr defaults datacenter", "!"]
24     for ai in range(k // 2):
25         lid = edge_agg_link_id(p, ei, ai)
26         lines += [f"interface Ethernet{ai + 1}", f" ip address {link_ip(lid, 0)}/31", "!"]
27     for j in range(k // 2):
28         lines += [f"interface Ethernet{k // 2 + j + 1}", f" ip address {host_gateway(p, ei, j)}/24", "!"]
29     lines += ["!", f"router bgp {edge_asn(p, ei)}"]
30     for ai in range(k // 2):
31         lid = edge_agg_link_id(p, ei, ai)
32         lines.append(f" neighbor {link_ip(lid, 1)} remote-as {agg_asn(p, ai)}")
33     for j in range(k // 2):
34         lines.append(f" network {host_network(p, ei, j)}")
35     return "\n".join(lines) + "\n"
36 # 生成汇聚交换机配置
37 def gen_agg(p, ai):
38     lines = ["frr defaults datacenter", "!"]
39     for ei in range(k // 2):
40         lid = edge_agg_link_id(p, ei, ai)
41         lines += [f"interface Ethernet{ei + 1}", f" ip address {link_ip(lid, 1)}/31", "!"]
42     for j in range(k // 2):
43         lid = agg_core_link_id(p, ai, j)
44         lines += [f"interface Ethernet{k // 2 + j + 1}", f" ip address {link_ip(lid, 0)}/31", "!"]
45     lines += ["!", f"router bgp {agg_asn(p, ai)}"]
46     for ei in range(k // 2):
47         lid = edge_agg_link_id(p, ei, ai)
48         lines.append(f" neighbor {link_ip(lid, 0)} remote-as {edge_asn(p, ei)}")
49     for j in range(k // 2):
50         lid = agg_core_link_id(p, ai, j)
51         core_idx = ai * (k // 2) + j
52         lines.append(f" neighbor {link_ip(lid, 1)} remote-as {core_asn(core_idx)}")
53     return "\n".join(lines) + "\n"
54 # 生成核心交换机配置
55 def gen_core(ci):
56     ai = ci // 2
57     j = ci % 2

```



```

58 lines = ["frr defaults datacenter", "!"]
59 for p in range(k):
60     lid = agg_core_link_id(p, ai, j)
61     lines += [f"interface Ethernet{p + 1}", f" ip address {link_ip(lid, 1)}/31", "!"]
62 lines += ["!", f"router bgp {core_asn(ci)}"]
63 for p in range(k):
64     lid = agg_core_link_id(p, ai, j)
65     lines.append(f" neighbor {link_ip(lid, 0)} remote-as {agg_asn(p, ai)}")
66 return "\n".join(lines) + "\n"
67 # 写入文件
68 for ci in range(k):
69     with open(os.path.join(config_dir, f"c{ci}.conf"), "w") as f:
70         f.write(gen_core(ci))
71
72 for p in range(k):
73     for ai in range(k // 2):
74         with open(os.path.join(config_dir, f"a{p}{ai}.conf"), "w") as f:
75             f.write(gen_agg(p, ai))
76     for ei in range(k // 2):
77         with open(os.path.join(config_dir, f"e{p}{ei}.conf"), "w") as f:
78             f.write(gen_edge(p, ei))
79 print(f"Generated configuration files in '{config_dir}' directory.")

```

运行之后，成功生成了 20 个配置文件，截图如下：

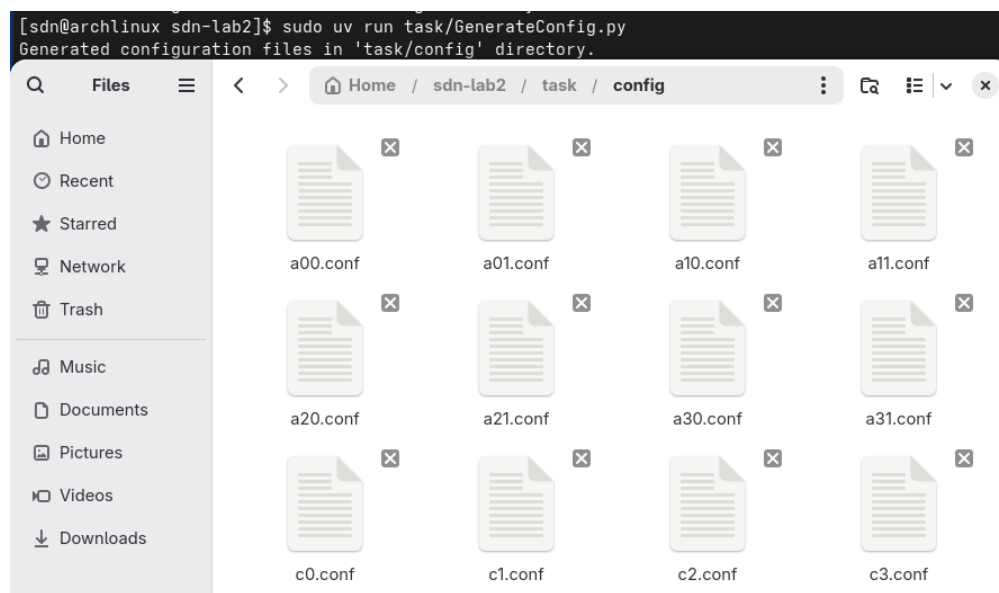


图 10: Generate Config File

我们点开 a00 汇聚，c0 核心，e00 边缘交换机的文件（从左到右），观察，符合我们的要求。截图如下：


```

mininet> h000 iperf -s &
mininet> h300 iperf -c 10.0.0.2 -P 4
-----
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[  4] local 10.3.0.2 port 59762 connected with 10.0.0.2 port 5001
[  3] local 10.3.0.2 port 59774 connected with 10.0.0.2 port 5001
[  1] local 10.3.0.2 port 59754 connected with 10.0.0.2 port 5001
[  2] local 10.3.0.2 port 59766 connected with 10.0.0.2 port 5001
[ ID] Interval          Transfer      Bandwidth
[  1] 0.0000-20.6580 sec  3.76 MBytes  1.53 Mbits/sec
[  2] 0.0000-20.6941 sec  3.21 MBytes  1.30 Mbits/sec
[  3] 0.0000-21.2574 sec  3.97 MBytes  1.57 Mbits/sec
[  4] 0.0000-21.2574 sec  3.91 MBytes  1.54 Mbits/sec
[SUM] 0.0000-21.2575 sec 14.8 MBytes  5.86 Mbits/sec

```

图 13: Throughput (without ECMP)

接下来，修改 GenerateConfig.py 的部分地方，在 router bgp 后面加上”bgp bestpath as-path multipath-relax”，删去旧的 config 文件，重新生成。之后再重新运行 FattreeTopo.py，重新测试 h000 和 h300 之间的吞吐量，结果如下：

```

mininet> h000 iperf -s &
-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
mininet> h300 iperf -c 10.0.0.2 -P 4
-----
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[  4] local 10.3.0.2 port 41618 connected with 10.0.0.2 port 5001
[  1] local 10.3.0.2 port 41608 connected with 10.0.0.2 port 5001
[  2] local 10.3.0.2 port 41592 connected with 10.0.0.2 port 5001
[  3] local 10.3.0.2 port 41594 connected with 10.0.0.2 port 5001
[ ID] Interval          Transfer      Bandwidth
[  4] 0.0000-14.2074 sec  5.15 MBytes  3.04 Mbits/sec
[  1] 0.0000-14.2394 sec  7.00 MBytes  4.13 Mbits/sec
[  2] 0.0000-15.0434 sec  6.28 MBytes  3.50 Mbits/sec
[  3] 0.0000-15.0433 sec  6.78 MBytes  3.78 Mbits/sec
[SUM] 0.0000-15.0434 sec 25.2 MBytes 14.1 Mbits/sec

```

图 14: Throughput (with ECMP)

我们发现，不开启 ECMP 时，吞吐量为 5.86Mbits/sec，开启之后为 14.1Mbits/sec。这说明 ECMP 生效了。在 Fattree 中，h300 到 h000 的跨 Pod 流量需要经过边缘 → 汇聚 → 核心 → 汇聚 → 边缘，存在多条等价路径。不开 ECMP 时 BGP 只选一条最优路径，其他链路闲置；开启后多条路径同时负载均衡，充分利

用了 Fat-tree 的冗余带宽。

5 ASN Multiplexing

现在在我们的配置中，每台交换机都公用一个 ASN（一共 20 个），但现实中 16 位 ASN 只有 1023 个私有号码，大型数据中心交换机数量是远超这个的，所以实际的做法就是在每层设备之间实现 ASN 的复用。

5.1 修改 ASN 方案

我们设定复用 ASN 的方案：

对于核心层的四台交换机：全都使用 65001；

对于每个 pod 的汇聚层：共用一个 ASN，分别为 65002、65003、65004、65005；

对于每个 pod 的边缘层：共用一个 ASN，分别为 65006、65007、65008、65009。

这样，一共使用了 9 个 ASN，减少了 ASN 的使用量。

我们修改一下 GenerateConfig.py，重新创建 ASNMultiplexingConfig.py，代码如下：

```
1 import os
2
3 k = 4
4 config_dir = "task/config"
5 os.makedirs(config_dir, exist_ok=True)
6 # ASN分配
7 def core_asn(ci):          return 65001          # 所有核心共用
8 def agg_asn(p, ai):        return 65002 + p      # 同Pod汇聚共用
9 def edge_asn(p, ei):       return 65006 + p      # 同Pod边缘共用
10 # 链路IP
11 def edge_agg_link_id(p, ei, ai): return p * 4 + ei * 2 + ai
12 def agg_core_link_id(p, ai, j):  return 16 + p * 4 + ai * 2 + j
13 def link_ip(lid, side):         return f"172.16.{lid}.{side}"
14 # 主机子网
15 def host_network(p, ei, j):
16     sid = ei * 2 + j
17     return f"10.{p}.{sid}.0/24"
18 def host_gateway(p, ei, j):
19     sid = ei * 2 + j
20     return f"10.{p}.{sid}.1"
21 # 生成边缘交换机配置
22 def gen_edge(p, ei):
23     lines = ["frr defaults datacenter", "!"]
24     for ai in range(k // 2):
25         lid = edge_agg_link_id(p, ei, ai)
26         lines += [f"interface Ethernet{ai + 1}", f" ip address {link_ip(lid, 0)}/31", "!"]
27     for j in range(k // 2):
28         lines += [f"interface Ethernet{k // 2 + j + 1}", f" ip address {host_gateway(p, ei, j)}/24", "!"]
29     lines += ["!", f"router bgp {edge_asn(p, ei)}", " bgp bestpath as-path multipath-relax"]
30     for ai in range(k // 2):
31         lid = edge_agg_link_id(p, ei, ai)
32         lines.append(f" neighbor {link_ip(lid, 1)} remote-as {agg_asn(p, ai)}")
33     for j in range(k // 2):
34         lines.append(f" network {host_network(p, ei, j)}")
35     return "\n".join(lines) + "\n"
```

```

36 # 生成汇聚交换机配置
37 def gen_agg(p, ai):
38     lines = ["frr defaults datacenter", "!"]
39     for ei in range(k // 2):
40         lid = edge_agg_link_id(p, ei, ai)
41         lines += [f"interface Ethernet{ei + 1}", f" ip address {link_ip(lid, 1)}/31", "!"]
42     for j in range(k // 2):
43         lid = agg_core_link_id(p, ai, j)
44         lines += [f"interface Ethernet{k // 2 + j + 1}", f" ip address {link_ip(lid, 0)}/31", "!"]
45     lines += ["!", f"router bgp {agg_asn(p, ai)}", " bgp bestpath as-path multipath-relax"]
46     for ei in range(k // 2):
47         lid = edge_agg_link_id(p, ei, ai)
48         lines.append(f" neighbor {link_ip(lid, 0)} remote-as {edge_asn(p, ei)}")
49     for j in range(k // 2):
50         lid = agg_core_link_id(p, ai, j)
51         core_idx = ai * (k // 2) + j
52         lines.append(f" neighbor {link_ip(lid, 1)} remote-as {core_asn(core_idx)}")
53     return "\n".join(lines) + "\n"
54 # 生成核心交换机配置
55 def gen_core(ci):
56     ai = ci // 2
57     j = ci % 2
58     lines = ["frr defaults datacenter", "!"]
59     for p in range(k):
60         lid = agg_core_link_id(p, ai, j)
61         lines += [f"interface Ethernet{p + 1}", f" ip address {link_ip(lid, 1)}/31", "!"]
62     lines += ["!", f"router bgp {core_asn(ci)}", " bgp bestpath as-path multipath-relax"]
63     for p in range(k):
64         lid = agg_core_link_id(p, ai, j)
65         lines.append(f" neighbor {link_ip(lid, 0)} remote-as {agg_asn(p, ai)}")
66     return "\n".join(lines) + "\n"
67 # 写入文件
68 for ci in range(k):
69     with open(os.path.join(config_dir, f"c{ci}.conf"), "w") as f:
70         f.write(gen_core(ci))
71
72 for p in range(k):
73     for ai in range(k // 2):
74         with open(os.path.join(config_dir, f"a{p}{ai}.conf"), "w") as f:
75             f.write(gen_agg(p, ai))
76     for ei in range(k // 2):
77         with open(os.path.join(config_dir, f"e{p}{ei}.conf"), "w") as f:
78             f.write(gen_edge(p, ei))
79
80 print(f"Generated configuration files in '{config_dir}' directory.")

```

5.2 连通性测试及问题分析

接下来，我们删除旧的 config，执行新的脚本，再测试 Fattree 拓扑的连通性，结果如下：

```

mininet> pingall
*** Ping: testing ping reachability
h000 -> h001 X X h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h001 -> h000 X X h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h010 -> X X h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h011 -> X X h010 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h100 -> h000 h001 h010 h011 h101 X X h200 h201 h210 h211 h300 h301 h310 h311
h101 -> h000 h001 h010 h011 h100 X X h200 h201 h210 h211 h300 h301 h310 h311
h110 -> h000 h001 h010 h011 X X h111 h200 h201 h210 h211 h300 h301 h310 h311
h111 -> h000 h001 h010 h011 X X h110 h200 h201 h210 h211 h300 h301 h310 h311
h200 -> h000 h001 h010 h011 h100 h101 h110 h111 h201 X X h300 h301 h310 h311
h201 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 X X h300 h301 h310 h311
h210 -> h000 h001 h010 h011 h100 h101 h110 h111 X X h211 h300 h301 h310 h311
h211 -> h000 h001 h010 h011 h100 h101 h110 h111 X X h210 h300 h301 h310 h311
h300 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h301 X X
h301 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 X X
h310 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 X X h311
h311 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 X X h310
*** Results: 13% dropped (208/240 received)

```

图 15: Connectivity Test

我们发现，每个 Pod 内同边缘交换机下的主机能通，但不同边缘交换机下的主机不通。下面我们分析分析为什么。执行 `e00 show ip bgp` 命令，看看 e00 缺失了哪些路由，结果如下：

Network	Next Hop	Metric	LocPrf	Weight	Path
*> 10.0.0.0/24	0.0.0.0(archlinux)	0		32768	i
*> 10.0.1.0/24	0.0.0.0(archlinux)	0		32768	i
*> 10.1.0.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65003 65007 i
*> 10.1.1.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65003 65007 i
*> 10.1.2.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65003 65007 i
*> 10.1.3.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65003 65007 i
*> 10.2.0.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65004 65008 i
*> 10.2.1.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65004 65008 i

图 16: Basic Instructions

*=	172.16.1.1(archlinux)			0	65002 65001 65004 65008 i
*> 10.2.2.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65004 65008 i
*> 10.2.3.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65004 65008 i
*> 10.3.0.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65005 65009 i
*> 10.3.1.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65005 65009 i
*> 10.3.2.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65005 65009 i
*> 10.3.3.0/24	172.16.0.1(archlinux)				
*=	172.16.1.1(archlinux)			0	65002 65001 65005 65009 i

图 17: Throughput Comparison

观察路由表，10.0.2.0/24 和 10.0.3.0/24 这两条路由是缺失的，而这两个是同 Pod 另一台边缘交换机 e01 的主机子网。这就导致了 ping 不通的情况出现。那么原因是什么呢？e00 和 e01 都是 ASN 65006。当 e01 把路由由 10.0.2.0/24 宣告给汇聚 a00 时，AS-Path 里带上了 65006。a00 再把这条路由传给 e00，e00 收到后发现 AS-Path 里有 65006——这是自己的 ASN！BGP 的防环机制认为这是一条环路，直接丢弃了。而跨 Pod 是没问题的，因为不同 Pod 的边缘 ASN 是不同的，就不会触发这样的防环路检测。

5.3 解决防环路检测问题

经过上面的分析，ping 不通的原因已经很明显了。那么我们考虑怎么修复。经过查阅资料，我们可以在边缘交换机上添加 `allowas-in`，告诉 BGP “允许 AS-Path 中包含自己 ASN 的路由”。另外值得注意的一点是，

同样的问题也会出现在核心层（四个 Pod 都是共用 65001 的）。我们模拟一下：比如路由从 a00 → c0(65001) → a10 → c1(65001)，c1 会发现 AS-Path 里已有 65001 而拒收。但 c1 本身已经直接从 a00 收到过同一条路由（a00 → c1），所以路由不会丢失（不影响连通性），但是这会导致 multipath 条数的减少，而实验要求里要求到了不能出现这一点。再考虑一下对于汇聚交换层会不会出现这种情况，考虑这样的一条路径：c0(65001) → a00(65002) → e00(65006) → a01(65002)，e00 作为 eBGP 路由器，会把从 a00 收到的路由转发给 a01。但 a01 收到后发现 AS-Path 里已经有了自己的 ASN，也会拒收，从而导致 multipath 条数的减少。所以在修改生成 config 文件的脚本时，要对三层交换机统一用 allowas-in：允许收到的路由 AS-Path 中包含自己的 ASN。

提出解决方案之后，我们来修改脚本，在生成配置的时候，在两端分别加上下面的语句就好：

```
1 lines.append(f" neighbor {link_ip(lid, 0)} allowas-in ")
2 lines.append(f" neighbor {link_ip(lid, 1)} allowas-in ")

```

修改好之后，运行脚本生成配置文件，然后检测连通性，结果如下：

```
mininet> pingall
*** Ping: testing ping reachability
h000 -> h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h001 -> h000 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h010 -> h000 h001 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h011 -> h000 h001 h010 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h100 -> h000 h001 h010 h011 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h101 -> h000 h001 h010 h011 h100 h110 h111 h200 h201 h210 h211 h300 h301 h310 h311
h110 -> h000 h001 h010 h011 h100 h101 h111 h200 h201 h210 h211 h300 h301 h310 h311
h111 -> h000 h001 h010 h011 h100 h101 h110 h200 h201 h210 h211 h300 h301 h310 h311
h200 -> h000 h001 h010 h011 h100 h101 h110 h111 h201 h210 h211 h300 h301 h310 h311
h201 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h210 h211 h300 h301 h310 h311
h210 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h211 h300 h301 h310 h311
h211 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h300 h301 h310 h311
h300 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h301 h310 h311
h301 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h310 h311
h310 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h311
h311 -> h000 h001 h010 h011 h100 h101 h110 h111 h200 h201 h210 h211 h300 h301 h310
*** Results: 0% dropped (240/240 received)

```

图 18: Connectivity Test

这样，我们就不影响 multipath 条数/不影响连通性的前提下，实现了 ASN 的复用。